Claude Code Workflow Playbook

From Lab to Real Work - Your First Week (August 2026 edition)

The 5-Minute Setup (any project)

1. `cd` into the project
2. Run `/init` - Claude scans and drafts a CLAUDE.md; trim to policy (< 200 lines):

```
# Project: [Name]
## Tech Stack [language, framework, key deps]
## Architecture [where things live]
## Conventions [naming, errors, tests]
## Never [things Claude must never do here]
```

3. Verify: "What conventions does this project follow?"
4. Check `/memory` - auto memory will accumulate build/test facts on its own; you own the rules, Claude owns the learnings.

When to Use What

You want to...	Do this	Why
Quick fix, single file	Ask naturally (Default mode)	Low risk, fast feedback
Multi-file change	**Shift+Tab -> Plan**, approve, then execute	Plan mode can't write - safe to think
Well-scoped grunt work	**Shift+Tab -> Auto** (with permission rules set)	Autonomy inside your fence
Explore unfamiliar code	"Explain @src/auth/ - what does this do?"	@ scopes the reading
Repetitive task (3+ times)	CLAUDE.md rule or a skill	Don't repeat yourself to Claude either
Risky (DB, deploy, delete)	Plan mode + review every diff + deny rules	The fence, not vibes

Mode cycle (Shift+Tab): Default -> AcceptEdits -> Plan -> Auto. Auto mode ships with two-layer safety (injection probe + transcript classifier) and rm -rf guards - but your deny/ask/allow rules are the real control. Deny is evaluated first.

The Prompt Quality Ladder

- **L1 vague:** "Add authentication" - works, wastes iterations
- **L2 specific (~70% first-try):** "Add JWT auth to @src/api/routes.js using jsonwebtoken. Protect /api/* except /api/health."
- **L3 contextual (~90%):** L2 + "secret in process.env.JWT_SECRET, 24h expiry, follow @src/middleware/errorHandler.js patterns, tests like @tests/api/."

The rule: whatever you'd correct in review - write it into the prompt instead.

Checkpoints & Rewind - the Recovery Playbook

Claude Code checkpoints continuously. **Double-tap Esc** opens the rewind menu.

Symptom	Restore
Bad file changes	**code**
Derailed/poisoned conversation	**conversation**
Both	**both**
Damage predates checkpoints (teammate's merge)	Rebuild: describe what's wrong + `@reference` files + what done looks like

Rewind beats arguing with contaminated context - always.

The Self-Healing Loop - When to Intervene

LET IT RUN: import errors, type mismatches, missing config, readable test failures.

INTERRUPT (Esc): rewriting files you didn't ask about; same error 3+ times; wrong architecture; burning tokens on something you know.

After interrupting: give a correction, not "try again." Or double-Esc and rewind - cheaper than correcting.

Context Discipline

- Same task = keep context; new task = `/clear`
- `/compact` around 50% usage - don't wait for degradation
- `/context` to see costs; `/cost` to see dollars; `/btw` for side questions that don't pollute
- `!command` runs shell and lets Claude react to the output - no copy-paste round trip
- Add Compact Instructions to CLAUDE.md so compaction preserves architecture decisions (see Templates handout)
- Multi-session work: end with "Write progress to HANDOFF.md - what you tried, what worked, what didn't" and start the next session from that file

Session Cost Sense

Use Sonnet 5 for normal implementation and Opus 5 for complex reasoning when available. Identify the meter before estimating cost: subscription seats use shared account allowances; API/cloud runs are token billed. Bound headless work with `--max-turns`, narrow tools, budgets, and telemetry.

Your First Week

Day	Do
1	/init + curated CLAUDE.md in ONE real repo; set deny/ask/allow via /permissions
2	First real ticket end-to-end: Plan mode -> execute -> /review -> PR
3	Write ONE skill for your most repeated review ask; eval it (5+5 prompts)
4	Wire the PostToolUse format hook + PreToolUse secret blocker from Lab 2B
5	Run `scripts/ci-review.sh` style headless check on a branch; demo to the team; propose the plugin

Weekly Hygiene

- `/doctor` - auto-fix issues; prune flagged unused skills/MCP servers; fix slow hooks
- Rerun skill evals after model updates
- Promote recurring auto-memory facts into CLAUDE.md; delete rules the model now does by default
- Check `/cost` trends; drop a model tier where quality allows

The Three-Layer Stack

1. **Policy** - CLAUDE.md + permission rules (deny -> ask -> allow)
2. **Automation** - skills, hooks, commands, plugins
3. **Verification** - graders, evals, /review, headless CI checks

Any single layer has gaps. All three together is an engineering system.